

From MERN basics to building with confidence

A practice-first roadmap for Node.js, TypeScript, Express, databases, infrastructure, and a final e-commerce backend.

START	FINISH	DAILY COMMITMENT
August 18, 2026	September 16, 2026	5-6 focused hours

You already know the basics. This month is about turning knowledge into evidence: a running server, a tested API, a cached endpoint, a published event, and a complete backend you can explain.

Use September 17 as a flex day if a build slips. The goal is not to finish every video; it is to finish the month with something real in your repository.

START HERE

The goal is confidence through repetition

You do not need a beginner's tour of backend development. You need a month of small, complete builds that make the next build feel possible.

THE OPERATING RULE

Follow the 30% learning / 70% building split. Learn just enough to make the next decision, then write the code yourself. When you get stuck, ask for an explanation of your code or error - not an entire project.

BY THE END OF DAY 30	YOU WILL BE ABLE TO
Build from zero	Open an empty folder, choose a structure, and create a working API without a line-by-line tutorial.
Understand tradeoffs	Compare MongoDB and PostgreSQL, synchronous calls and events, and database reads and cache hits.
Ship production habits	Add validation, security, testing, logging, health checks, Docker, and documentation.
Finish a portfolio piece	Deliver an e-commerce/order-management backend with auth, products, carts, orders, Redis, Kafka, and tests.

Technology sequence

JavaScript review -> TypeScript -> Node.js -> Express -> REST -> Auth -> MongoDB -> PostgreSQL/SQL -> Prisma -> Redis -> Kafka -> Docker -> Testing -> Production architecture

EVERY 2-3 DAYS

Have something running. Replace statements like 'I finished Redis' with proof like 'I added caching and can show a cache hit.'

YOUR DAILY SYSTEM

A 5-6 hour routine that produces code

Protect the building block of the day. If time is short, shrink the learning session before you shrink the hands-on session.

BLOCK	TIME	WHAT TO DO
1. Learn	90 min	Read docs, watch a focused lesson, and write 5-10 notes in your own words.
2. Build	150 min	Implement the day's deliverable from a blank file or your current project. No copy-paste project tutorial.
3. Debug	60 min	Use logs, small experiments, documentation, and targeted questions to remove blockers.
4. Record	30 min	Commit your work and write what you learned, what you built, and what still feels unclear.

The end-of-day note

WRITE THIS EVERY DAY

Today I learned: ...

What I built: ...

What broke and how I fixed it: ...

What I still cannot explain: ...

Tomorrow I will start by: ...

DAILY CHECK	PROMPT
☐	I wrote code before I watched more content.
☐	I ran the feature locally and tested at least one failure case.
☐	I committed a small, understandable change.
☐	I wrote a short note in my own words.

WEEK 1 / DAYS 1-7

Foundation and confidence

Move quickly through familiar basics and prove that you can start from an empty folder.

DAY 1

Tue, Aug 18

Node.js + TypeScript setup**LEARN / REVISE**

Revise runtime, V8, event loop, async/await, modules, npm, env vars. Create a strict TS project.

BUILD

Build a CLI Notes app with add, list, and delete commands using fs/path.

DELIVERABLE

Working TypeScript CLI and a short note explaining the event loop.

DAY 2

Wed, Aug 19

Node.js under the hood**LEARN / REVISE**

Study EventEmitter, buffers, streams, file system, raw http, and how Node handles concurrent requests.

BUILD

Build an HTTP server without Express: GET /users, GET /products, POST /users.

DELIVERABLE

Raw HTTP server with status codes and JSON responses.

DAY 3

Thu, Aug 20

Express.js essentials**LEARN / REVISE**

Routing, controllers, middleware, request/response, params, query strings, JSON bodies, and status codes.

BUILD

Create /users, /products, and /orders routes with in-memory data.

DELIVERABLE

A small Express API you can explain line by line.

DAY 4

Fri, Aug 21

Backend architecture**LEARN / REVISE**

Practice the flow: Route -> Controller -> Service -> Repository/Model. Keep responsibilities separate.

BUILD

Refactor the API into src/routes, controllers, services, models, middleware, config, and app.ts.

DELIVERABLE

A clean CRUD API with a predictable folder structure.

DAY 5

Sat, Aug 22

Errors + validation**LEARN / REVISE**

Global error middleware, custom errors, status codes, async error handling, and Zod validation.

BUILD

Add POST /users with validated name, email, password, and age fields.

DELIVERABLE

Consistent error responses and validation tests by hand.

DAY 6

Sun, Aug 23

Authentication with JWT**LEARN / REVISE**

bcrypt, access tokens, refresh tokens, cookies, auth middleware, roles, and authorization.

BUILD

Build register, login, refresh, me, and logout endpoints.

DELIVERABLE

Protected routes plus a written token flow diagram.

WEEK 1 / CONTINUED

Finish the week with evidence

Complete the remaining day, then pause for a short review before moving on.

DAY 7Mon, Aug 24

Mini Project 1: Task API

LEARN / REVISE

Combine the week's concepts. Use documentation when stuck; do not follow a full tutorial.

BUILD

Build register/login, task CRUD, filtering, pagination, auth, authorization, validation, and errors.

DELIVERABLE

A working Task Management API and README with setup steps.

MILESTONE

You can now create and protect a small API from scratch. Keep the Task API running; it becomes your reference project for the rest of the month.

Weekly review

CHECK	PROMPT
Build without a tutorial	Which part of the API can you recreate tomorrow from memory?
Error flow	Where does a bad request become a useful response?
Auth gap	Which JWT or cookie detail still feels fuzzy?

WEEK 2 / DAYS 8-14

Databases and data design

Use MongoDB and PostgreSQL to understand two different ways of modeling real systems.

DAY 8

Tue, Aug 25

MongoDB fundamentals**LEARN / REVISE**

Documents, collections, BSON, CRUD, indexes, aggregation, embedding vs referencing, and relationships.

BUILD

Connect MongoDB to your API and model users, tasks, and tags.

DELIVERABLE

A database-backed API with at least one useful index.

DAY 9

Wed, Aug 26

Mongoose**LEARN / REVISE**

Schemas, models, validation, hooks, populate, indexes, and transaction basics.

BUILD

Model User, Post, and Comment with references and a populated response.

DELIVERABLE

Mongoose schemas plus a relationship query you can explain.

DAY 10

Thu, Aug 27

Mini Project 2: Blog API**LEARN / REVISE**

Use MongoDB as a design exercise, not just a CRUD exercise.

BUILD

Build auth, posts, likes, comments, follows, search, and pagination.

DELIVERABLE

A working Blog API with sample seed data.

DAY 11

Fri, Aug 28

SQL fundamentals**LEARN / REVISE**

SELECT, INSERT, UPDATE, DELETE, WHERE, ORDER BY, GROUP BY, HAVING, LIMIT/OFFSET, and JOINS.

BUILD

Write queries for users, products, orders, and order_items. Include inner and left joins.

DELIVERABLE

A saved SQL practice file with 15 queries and explanations.

DAY 12

Sat, Aug 29

PostgreSQL data modeling**LEARN / REVISE**

Tables, primary/foreign keys, constraints, indexes, relationships, and transactions.

BUILD

Create users, products, orders, and order_items with realistic constraints.

DELIVERABLE

A relational schema and migration script.

DAY 13

Sun, Aug 30

Prisma**LEARN / REVISE**

Schema, models, relations, migrations, Prisma Client, queries, and transactions.

BUILD

Connect Prisma to PostgreSQL and rebuild one API feature with a transaction.

DELIVERABLE

Prisma schema, migrations, and one tested transaction.

WEEK 2 / CONTINUED

Finish the week with evidence

Complete the remaining day, then pause for a short review before moving on.

DAY 14Mon, Aug 31

Relational mini-commerce

LEARN / REVISE

Compare MongoDB and SQL tradeoffs: consistency, joins, relationships, and query shape.

BUILD

Build products, cart, orders, order items, and payment records with PostgreSQL.

DELIVERABLE

Mini e-commerce API with order creation in a transaction.

MILESTONE

You can model the same business idea in a document database and a relational database. That comparison is more valuable than memorizing either tool in isolation.

Weekly review

CHECK	PROMPT
Database choice	When would you choose MongoDB? When would you choose PostgreSQL?
Transaction boundary	Which operation must succeed or fail as one unit?
Query skill	Which SQL query or index did you understand better this week?

WEEK 3 / DAYS 15-21

Infrastructure and events

Turn a working API into a more resilient system with caching, events, and local infrastructure.

DAY 15

Tue, Sep 1

Redis fundamentals**LEARN / REVISE**

Key-value storage, TTL, caching, sessions, Pub/Sub, and core data structures.

BUILD

Add Redis and measure the difference between a database request and a cache hit.

DELIVERABLE

A before/after latency note and a working Redis connection.

DAY 16

Wed, Sep 2

Caching in practice**LEARN / REVISE**

Cache-aside pattern, cache keys, TTL choices, and invalidation after writes.

BUILD

Cache product lists and user profiles. Invalidate after create/update/delete.

DELIVERABLE

Products served from Redis on repeated requests.

DAY 17

Thu, Sep 3

Redis for reliability**LEARN / REVISE**

Rate limiting, sessions, distributed locks, Pub/Sub, and queues.

BUILD

Build a 100 requests/minute API limiter and document the chosen key strategy.

DELIVERABLE

A tested rate limiter with 429 responses.

DAY 18

Fri, Sep 4

Kafka fundamentals**LEARN / REVISE**

Producer, broker, topic, partition, consumer, consumer group, and offset.

BUILD

Run Kafka locally and publish a simple OrderCreated event.

DELIVERABLE

A producer and consumer that can be restarted safely.

DAY 19

Sat, Sep 5

Kafka with Node.js**LEARN / REVISE**

Event schemas, consumer error handling, retries, and the difference between sync calls and events.

BUILD

Build Order Service -> Kafka -> Email Service for an order-created notification.

DELIVERABLE

An event-driven flow with a reproducible demo.

DAY 20

Sun, Sep 6

Event-driven services**LEARN / REVISE**

Think in events and ownership. Keep services small enough to understand.

BUILD

Add Payment, Notification, and Analytics consumers for OrderCreated.

DELIVERABLE

Three consumers reading one topic, with clear event names.

WEEK 3 / CONTINUED

Finish the week with evidence

Complete the remaining day, then pause for a short review before moving on.

DAY 21

Mon, Sep 7

Docker + Compose

LEARN / REVISE

Images, containers, Dockerfiles, volumes, networks, and service configuration.

BUILD

Run Node, PostgreSQL, Redis, and Kafka using Docker Compose.

DELIVERABLE

One command starts the full local backend environment.

MILESTONE

Your local backend environment can now behave like a small system: cached reads, rate limits, events, and services started together.

Weekly review

CHECK	PROMPT
Cache	What makes a cache entry stale, and how do you invalidate it?
Consumer down	What happens to an event if a consumer is unavailable?
Compose	Can you explain what each local service is doing?

WEEK 4 / DAYS 22-30

Production habits and final build

Harden, test, document, and ship one complete e-commerce/order-management backend.

DAY 22

Tue, Sep 8

Security essentials**LEARN / REVISE**

CORS, Helmet, rate limiting, JWT safety, hashing, input validation, SQL/NoSQL injection, XSS, CSRF, and secrets.

BUILD

Harden the mini-commerce API and add a security checklist to the README.

DELIVERABLE

Security middleware enabled and secrets removed from source control.

DAY 23

Wed, Sep 9

Testing**LEARN / REVISE**

Unit, integration, and API testing with Vitest or Jest plus Supertest.

BUILD

Test register, login, create product, create order, and get order.

DELIVERABLE

A repeatable test command with meaningful failure messages.

DAY 24

Thu, Sep 10

Logging + operations**LEARN / REVISE**

Pino/Winston, request IDs, error logs, health checks, and graceful shutdown.

BUILD

Add /health, structured logs, request IDs, and clean database shutdown.

DELIVERABLE

Logs that help you diagnose one failing request.

DAY 25

Fri, Sep 11

API design**LEARN / REVISE**

REST principles, versioning, pagination, filtering, sorting, search, status codes, and idempotency.

BUILD

Design /api/v1/users, /api/v1/products, and /api/v1/orders with consistent responses.

DELIVERABLE

An API contract in the README, including examples.

DAY 26

Sat, Sep 12

Architecture review**LEARN / REVISE**

Layered architecture, clean architecture basics, dependency injection, and separation of concerns.

BUILD

Refactor toward Controller -> Service -> Repository -> Database without over-engineering.

DELIVERABLE

A one-page architecture decision record.

DAY 27

Sun, Sep 13

Final project: foundation**LEARN / REVISE**

Start from an empty folder and define scope, schema, environment, and API conventions.

BUILD

Set up TypeScript, Express, Prisma/PostgreSQL, Docker, auth, roles, and product CRUD.

DELIVERABLE

Final project boots locally with auth and products working.

WEEK 4 / CONTINUED

Finish the week with evidence

Complete the remaining day, then pause for a short review before moving on.

DAY 28 Mon, Sep 14	DAY 29 Tue, Sep 15
Final project: cart + orders	Final project: production layer
LEARN / REVISE Transactions, consistent order state, ownership checks, and clean error handling.	LEARN / REVISE Caching, invalidation, Kafka events, tests, logging, health checks, and rate limiting.
BUILD Build cart add/remove/update, checkout, order history, order detail, and cancel order.	BUILD Add Redis product caching, OrderCreated events, consumers, core tests, and observability.
DELIVERABLE A complete order flow from product to saved order.	DELIVERABLE A production-shaped backend with a recorded demo flow.

DAY 30 Wed, Sep 16
Final project: ship + reflect
LEARN / REVISE Polish what you built. Review tradeoffs, failure cases, and the next skills to deepen.
BUILD Write README, API examples, architecture diagram, seed script, Docker instructions, and a short demo.
DELIVERABLE A portfolio-ready backend and a personal confidence retrospective.

MILESTONE

You have a complete, explainable backend to show. Take September 17 as a flex day for cleanup, a second demo, or a deeper review of the hardest concept.

Weekly review

CHECK	PROMPT
Definition of done	Which final-project flow can another person run today?
Highest risk	What is the one gap you would fix before sharing the repo?
Next loop	What will you build in the next 30 days to deepen this foundation?

CAPSTONE

Final project: e-commerce and order management backend

The final project is intentionally small enough to finish and rich enough to demonstrate real backend judgment.

LAYER	CHOSEN TOOLS
Application	TypeScript, Node.js, Express.js, Zod
Data	PostgreSQL, Prisma, migrations, transactions
Infrastructure	Redis, Kafka, Docker Compose
Quality	JWT auth, rate limiting, tests, structured logs, health checks

Required API surface

AREA	MINIMUM SCOPE
Auth	POST /api/v1/auth/register login refresh logout; GET /me; roles
Products	Create, read, update, delete, search, filter, sort, pagination
Cart	Add item, remove item, update quantity, get current cart
Orders	Create order, list orders, get order detail, cancel order
Events	OrderCreated, PaymentProcessed, OrderCancelled

Suggested flow

Client -> Express routes -> controllers -> services -> repositories -> PostgreSQL. Redis sits beside the database for cache and rate limiting. Kafka carries order events to payment, notification, and analytics consumers.

DEFINITION OF DONE

A fresh clone can start the stack, run migrations and seed data, authenticate, create a product, add it to a cart, place an order, observe an OrderCreated event, and run the test suite by following your README.

CAPSTONE BLUEPRINT

Keep the architecture clear enough to explain

The point is not to build a distributed company. The point is to learn where responsibilities belong and how a request moves through a system.

LAYER	RESPONSIBILITY
HTTP layer	Routes and controllers translate requests into validated service calls.
Business layer	Services own rules such as stock checks, order totals, and cancellation policy.
Data layer	Repositories and Prisma own queries, migrations, and transactions.
Async layer	Kafka consumers react to named events without tightly coupling services.
Operations layer	Redis, logs, health checks, tests, and Docker make the system observable and repeatable.

Core data model

ENTITY	STARTING FIELDS
User	id, email, passwordHash, role, createdAt
Product	id, name, description, price, stock, createdAt
Cart / CartItem	userId; productId; quantity
Order / OrderItem	userId, status, total; product snapshot, quantity, price
Payment	orderId, status, providerRef, processedAt

Event contracts

EVENT	PAYLOAD STARTER
OrderCreated	orderId, userId, total, occurredAt
PaymentProcessed	orderId, paymentId, status, occurredAt
OrderCancelled	orderId, reason, occurredAt

ARCHITECTURE QUESTION

For every feature, ask: Which layer owns this decision? What happens if the database fails? Can I test it without starting every service?

TRACK THE MONTH

Milestones that make progress visible

Confidence grows when you can point to a working artifact. Mark the build, not just the lesson.

CHECKPOINT	ARTIFACT	EVIDENCE
Day 2	Node proof	Raw HTTP server works and returns JSON.
Day 7	Task API	Auth-protected CRUD API with validation and pagination.
Day 14	Data design	Mini commerce API backed by PostgreSQL and Prisma.
Day 21	Local system	Node + Postgres + Redis + Kafka run with Compose.
Day 26	Production shape	Security, tests, logging, API contract, and architecture notes.
Day 30	Portfolio build	Documented e-commerce backend with a demo flow.

30-day tracker

DAY	DATE	LEARN	BUILD	COMMIT	REFLECT
01	Tue, Aug 18	☐ Learn	☐ Build	☐ Commit	☐ Reflect
02	Wed, Aug 19	☐ Learn	☐ Build	☐ Commit	☐ Reflect
03	Thu, Aug 20	☐ Learn	☐ Build	☐ Commit	☐ Reflect
04	Fri, Aug 21	☐ Learn	☐ Build	☐ Commit	☐ Reflect
05	Sat, Aug 22	☐ Learn	☐ Build	☐ Commit	☐ Reflect
06	Sun, Aug 23	☐ Learn	☐ Build	☐ Commit	☐ Reflect
07	Mon, Aug 24	☐ Learn	☐ Build	☐ Commit	☐ Reflect
08	Tue, Aug 25	☐ Learn	☐ Build	☐ Commit	☐ Reflect
09	Wed, Aug 26	☐ Learn	☐ Build	☐ Commit	☐ Reflect
10	Thu, Aug 27	☐ Learn	☐ Build	☐ Commit	☐ Reflect
11	Fri, Aug 28	☐ Learn	☐ Build	☐ Commit	☐ Reflect
12	Sat, Aug 29	☐ Learn	☐ Build	☐ Commit	☐ Reflect
13	Sun, Aug 30	☐ Learn	☐ Build	☐ Commit	☐ Reflect
14	Mon, Aug 31	☐ Learn	☐ Build	☐ Commit	☐ Reflect
15	Tue, Sep 1	☐ Learn	☐ Build	☐ Commit	☐ Reflect
16	Wed, Sep 2	☐ Learn	☐ Build	☐ Commit	☐ Reflect
17	Thu, Sep 3	☐ Learn	☐ Build	☐ Commit	☐ Reflect

DAY	DATE	LEARN	BUILD	COMMIT	REFLECT
18	Fri, Sep 4	☐ Learn	☐ Build	☐ Commit	☐ Reflect
19	Sat, Sep 5	☐ Learn	☐ Build	☐ Commit	☐ Reflect
20	Sun, Sep 6	☐ Learn	☐ Build	☐ Commit	☐ Reflect
21	Mon, Sep 7	☐ Learn	☐ Build	☐ Commit	☐ Reflect
22	Tue, Sep 8	☐ Learn	☐ Build	☐ Commit	☐ Reflect
23	Wed, Sep 9	☐ Learn	☐ Build	☐ Commit	☐ Reflect
24	Thu, Sep 10	☐ Learn	☐ Build	☐ Commit	☐ Reflect
25	Fri, Sep 11	☐ Learn	☐ Build	☐ Commit	☐ Reflect
26	Sat, Sep 12	☐ Learn	☐ Build	☐ Commit	☐ Reflect
27	Sun, Sep 13	☐ Learn	☐ Build	☐ Commit	☐ Reflect
28	Mon, Sep 14	☐ Learn	☐ Build	☐ Commit	☐ Reflect
29	Tue, Sep 15	☐ Learn	☐ Build	☐ Commit	☐ Reflect
30	Wed, Sep 16	☐ Learn	☐ Build	☐ Commit	☐ Reflect

FINISH STRONG

Your review and confidence checklist

Use this on September 16, or September 17 if you need the flex day. Be honest; the purpose is to choose your next practice loop.

DONE	CONFIDENCE CHECK
☐	I can create a TypeScript + Express API from an empty folder.
☐	I can explain the request path from route to database.
☐	I can implement validation, consistent errors, and protected routes.
☐	I can choose when MongoDB or PostgreSQL is a better fit.
☐	I can explain a cache hit, invalidation, and a rate limit.
☐	I can explain a Kafka topic, partition, consumer group, and offset.
☐	I can start my local services with Docker Compose.
☐	I have tests for the most important business flows.
☐	I can read my logs and find the failing request.
☐	My README lets another person run and understand the project.

Retrospective prompts

PROMPT	YOUR NOTES
Most useful build	Which artifact taught you the most? Why?
Hardest concept	What still feels fuzzy? Name one small experiment for next week.
Best debugging moment	What did you try before asking for help? What worked?
Next 30 days	Choose one: deeper SQL, system design, background jobs, deployment, or observability.

You are not trying to become perfect in 30 days. You are building the habit of opening an empty folder, making a reasonable decision, and staying with the problem until something works.

Start today. Day 1 is not preparation for the work - Day 1 is the work.